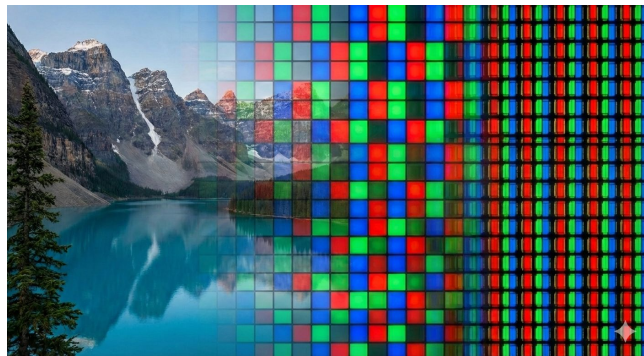


UT4 - Tratamiento de imagen



Jon Apesteguía
2025-2026

Objetivos

- Entender qué es realmente una imagen digital (Muestreo y Cuantificación).
- Visualizar la imagen como una estructura de datos (Matriz).
- Comprender el sistema de coordenadas en computación.
- Tipos de imágenes y profundidad de color.
- Relaciones entre píxeles (Vecindad).

¿Qué es una Imagen Digital?

Definición Matemática: Función $f(x, y)$ donde en cada posición hay un valor de intensidad.

Del Mundo Real al Digital:

- El mundo es continuo (infinitos detalles) → El ordenador es discreto (valores finitos).

Los 2 Procesos de Digitalización:

1. Muestreo (Sampling) → Resolución:

- Decidimos cuántos píxeles tendrá la rejilla.
- *Ejemplo:* 1920x1080 vs 320x240.

2. Cuantificación (Quantization) → Profundidad de Color:

- Decidimos cuántos valores/colores puede tener cada píxel.
- *Ejemplo:* ¿2 colores (Blanco/Negro) o 16 millones (True Color)?

La Unidad Mínima: El Píxel

Origen: "Picture Element". Es el punto más pequeño direccionable.

Atributos:

1. **Posición:** ¿Dónde está? (Fila, Columna).
2. **Valor:** ¿Cómo es?
 - Escala de grises: Un número de brillo (0-255).
 - Color: Un vector de 3 números [R, G, B].

La Imagen como Matriz

Concepto Fundamental: Para el ordenador, **no existe la "foto", solo existen números.**

Representación Matemática: Una imagen es una matriz A de dimensiones $M \times N$.

$$A = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,0} & a_{m,1} & \cdots & a_{m,n} \end{bmatrix}$$

Cada celda de la matriz es un píxel.

Sistema de Coordenadas

El error nº1 en Visión Artificial: Confundir Matemáticas con Programación.

Matemáticas (Plano Cartesiano):

- Origen (0,0): Abajo a la izquierda.
- Ejes: x horizontal, y vertical (hacia arriba).

Informática (Matrices/Imágenes):

- **Origen (0,0): Esquina SUPERIOR Izquierda.**
- **Ejes:**
 1. Eje 0 (Vertical hacia abajo) → **FILAS (y)**.
 2. Eje 1 (Horizontal derecha) → **COLUMNAS (x)**.

Regla de Oro en NumPy:

- Accedemos siempre como: `imagen[FILA, COLUMNA]`
- Equivalente a: `imagen[y, x]` (Al revés que en geometría)

Profundidad de Color: ¿Cuántos lápices tenemos?

Concepto: La profundidad de color (Bit Depth) define la cantidad de valores posibles que puede tomar un solo píxel. Es el resultado directo de la **Cuantificación**.

El Estándar: Escala de Grises (uint8)

- En IA y Visión Artificial, la mayoría de algoritmos trabajan en blanco y negro para ahorrar cómputo.
- **Tipo de dato:** uint8 (Unsigned Integer 8-bit).
- **Cálculo:** $2^8 = 256$ niveles de gris.

La Escala:

- **0** → Negro Absoluto (Ausencia de señal).
- **255** → Blanco Puro (Saturación total).
- *Intermedios:* Grises (ej. 128 es gris medio).

El error clásico: Desbordamiento de Bits

El Problema: NumPy optimiza la memoria usando 8 bits. Funciona como un reloj: al llegar al tope, vuelve a empezar.

Ejemplo Práctico:

- Imagina un píxel muy brillante: **Valor 250**.
- Queremos aumentar su brillo sumando **10**.

La Trampa:

- **Matemáticas:** $250 + 10 = 260$ (Blanco saturado).
- **Informática (uint8):** $260 \pmod{256} = 4$ (Negro casi puro).

Consecuencia Visual:

- Al sumar dos imágenes, aparecen manchas negras ("ruido") en las zonas más brillantes.

Solución: Convertir temporalmente a float o int16 para operar, y luego volver a uint8.

Más allá del Gris: Color y Binarias

Imágenes a Color (RGB):

- Ya no es una matriz 2D, es un **Tensor 3D**.
- **Dimensiones:** (Alto, Ancho, 3).
- **El Píxel:** Es un vector [R, G, B].
 - Rojo puro: [255, 0, 0]
 - Amarillo: [255, 255, 0] (Rojo + Verde).
- **Coste:** Ocupa el triple de memoria RAM.

Imágenes Binarias (Máscaras):

- **Valores:** Solo **0** (Fondo) y **1** (Objeto).
- **Uso:** Detección y Segmentación (ej. ¿Hay un coche aquí? Sí/No).

Resumen de tipos

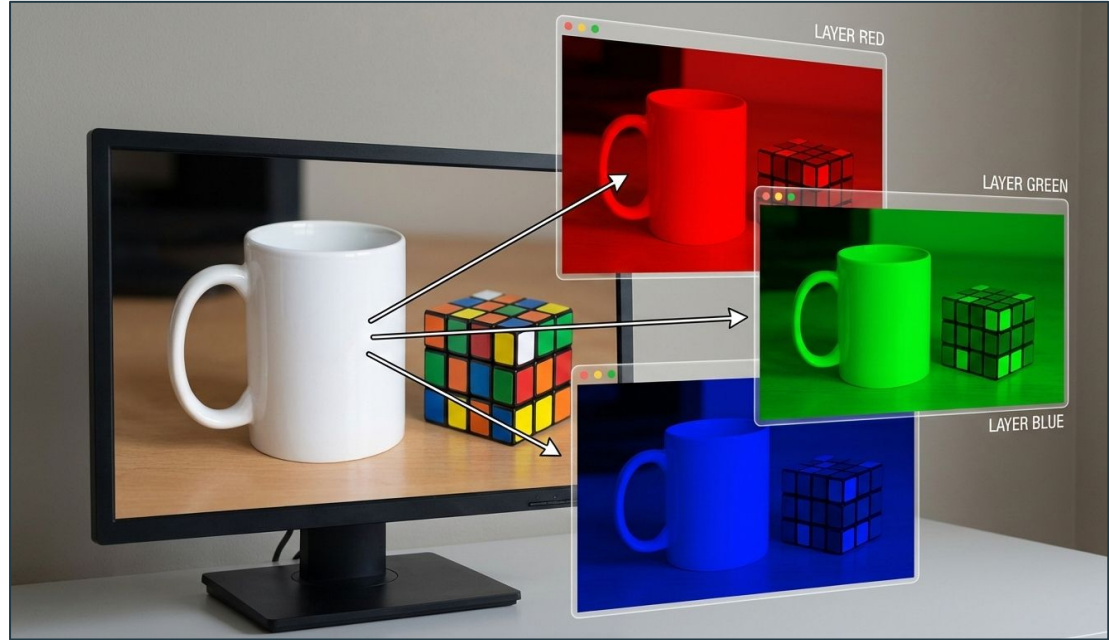
Tipo de Imagen	Canales	Estructura NumPy (.shape)	Rango de Valores	Tipo de Dato (dtype)	Uso Principal
Escala de Grises	1	(Alto, Ancho)	0 a 255	uint8	Procesamiento general, detección de bordes, reconocimiento facial.
Color (RGB)	3	(Alto, Ancho, 3)	0 a 255 (x3)	uint8	Visualización para humanos, segmentación por color.
Binaria	1	(Alto, Ancho)	0 ó 1	bool o uint8	Máscaras, resultado de umbralización, detección de movimiento.
Matemática	1 o 3	(Alto, Ancho, ...)	0.0 a 1.0	float32	Operaciones complejas (divisiones, redes neuronales) para evitar <i>overflow</i> .

Canales de Color

Modelo RGB: Modelo aditivo.

Descomposición:

- Una imagen RGB son **tres matrices de escala de grises apiladas.**



Relaciones entre Píxeles

Importancia: Para procesar una imagen (filtros, bordes), miramos un píxel y a sus vecinos.

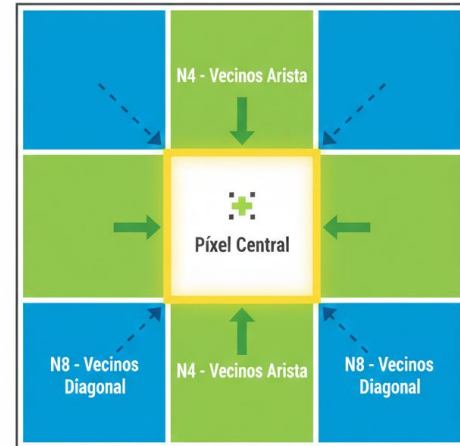
Vecindad-4 (N4):

- Los que comparten una arista (Arriba, Abajo, Izquierda, Derecha).
- Coordenadas: $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, $(x, y - 1)$

Vecindad-8 (N8):

- Incluye N4 + las 4 diagonales.

Relaciones entre Píxeles: Vecindad N4 y N8



Herramientas de Python

NumPy (np):

- El motor de cálculo.
- La imagen es un array de NumPy.
- Operaciones: Slicing, Suma, Resta.

Matplotlib (plt):

- El visualizador.
- `plt.imread()`: Cargar imagen a array.
- `plt.imshow()`: Pintar array como imagen.

Resumen de Comandos Clave

Cargar: `img = plt.imread('foto.jpg')`

Ver propiedades:

- `img.shape` → (Alto, Ancho, Canales)
- `img.dtype` → `uint8` o `float32`
- `img.min()`, `img.max()` → Rango dinámico

Recortar (Crop): `recorte = img[0:100, 0:100]`